R

RELAY AUTOMATION

COMPLETE PROJECT REFERENCE

Customer implementation, made clearer.

A comprehensive guide to Relay's purpose, public pages, product modules, user workflows, data model, integrations, safeguards, and current launch status.

Repository-based product and technical reference

Prepared 27 October 2026 | Reviewed repository commit: e53033c

For product understanding and planning. Not a customer contract, legal advice, security certification, or guarantee of production availability.

01 / HOW TO USE THIS DOCUMENT

Scope, status, and contents

This document describes Relay as represented in the source repository at the stated review point. It explains the visible product and the supporting systems, and separates shipped code from unverified deployment and business arrangements.

> **Privacy boundary:** this reference intentionally contains no API keys, passwords, database contents, account information, customer records, invitation tokens, or real customer examples. “All Relay information” here means product and project information, not confidential runtime data.

> **Verified in this review:** the repository was clean at commit e53033c; the automated Node test suite passed 4 of 4 tests; and `npm audit --omit=dev` reported zero production dependency vulnerabilities. These checks do not prove a live deployment or successful Docker build.

Contents

1. Relay at a glance
2. Audience, problem, and product boundaries
3. Public website and inquiry paths
4. Product workspace modules
5. End-to-end onboarding workflow
6. Roles, access, and customer participation
7. Data model and information categories
8. Architecture and integrations
9. Security and privacy controls
10. Testing and quality evidence
11. Deployment and operations
12. Current gaps, risks, and readiness verdict
13. Repository map and glossary

02 / AT A GLANCE

What Relay is

Relay is a standalone customer-implementation workspace for B2B SaaS teams. It connects reusable onboarding processes, customer-specific task plans, customer setup information, and readiness gaps in one place.

BUYER

B2B SaaS company

A business seeking a repeatable, visible way to coordinate multi-step customer onboarding or implementation.

DAILY USERS

Implementation team

Implementation, onboarding, or professional-services staff who own tasks and coordinate customer inputs.

PARTICIPANTS

Customer stakeholders

People at the SaaS company's customer who submit requested setup information through a private link.

Core value proposition

- Turn the team's onboarding knowledge into an ordered, checkable process.
- Reuse a verified snapshot so each customer implementation starts consistently.
- Track task status, ownership, dates, blockers, and prerequisites for each customer.
- Request typed customer data and identify missing or invalid required responses.
- Make next actions and readiness gaps visible without implying that readiness equals technical certification.

> **Product boundary:** Relay operates on data and workflows created inside Relay. It does not synchronize with external project-management tools or configure the customer's software, network, DNS, identity provider, or infrastructure.

03 / PUBLIC WEBSITE

Marketing, conversion, and trust pages

Page/block	Purpose and content	Important boundary
Home `index.html`	Positioning, illustrative onboarding stages, six capability cards, demo and feature calls to action.	Preview is illustrative, not customer evidence.
About `about.html`	Explains the onboarding coordination problem, buyer, daily team, customer participants, and standalone workspace approach.	Does not establish verified market demand.
Solutions `features-services.html`	Describes process mapping, workflow execution, templates, handover checks, customer information, and readiness gaps.	No external project-tool sync is offered.
Pricing `pricing.html`	One plan structure; quote-based pricing and a pre-purchase written-terms flow.	No fixed price, checkout, or subscription billing is implemented.
Demo `demo.html`	Explains a focused product walkthrough and collects demo requests.	Visitors are told not to provide confidential customer data.
Contact `contact.html`	Sales/contact inquiry form with name, company, work email, international phone, and details.	A monitored support email/channel and operator identity are not confirmed.
Privacy `privacy.html`	Summarizes account, workspace, customer submission, inquiry, AI-help, and hosting information.	Host, region, operator, retention, and applicable legal obligations need deployment-specific review.
Terms `terms.html`	Describes account responsibilities, acceptable use, service boundaries, and fields that a quote/order must cover.	It is not a completed customer-specific service agreement.
Customer setup `customer.html`	External customer page for viewing and submitting requested implementation inputs.	Anyone holding a valid private link may access that request; links must be shared carefully.
Inquiry dashboard `dashboard.html`	Separate internal dashboard for stored sales/demo inquiries.	It is distinct from the SaaS workspace; protect its bearer API token.

> **Public copy is not operational proof:** pages and integration hooks exist in code, but email, messaging, AI, HTTPS, domain ownership, monitoring, and real customer support have not been verified in production.

04 / PRODUCT WORKSPACE

Every major product module

01 / OVERVIEW

Workspace overview

Summarizes implementation activity, templates, and attention indicators. Offers a getting-started sequence, actionable empty states, refresh, and routes into implementation work.

Signals include missing information, overdue work, blocked tasks, unassigned tasks, and unmet prerequisites.

02 / PROCESS MAP

Design and check a workflow

Create processes and ordered steps. Capture instructions, stage, responsible role, assigned member, due-date offset, and prerequisite relationships. Run a setup check and view a handover summary before saving.

A check is a workflow completeness aid, not a customer implementation certification.

03 / TEMPLATES

Reuse versioned snapshots

Save a process that passes its check as a template. Inspect template history, restore a past version, or refresh from a checked process.

Existing implementations are their own task plans; template updates are intended for future starts.

04 / IMPLEMENTATIONS

Run customer work

Start from a template and create customer-specific tasks. Track status, stage, owner/member, due date, blocker reason, next action, prerequisites, and overall attention indicators.

Find records by customer/template, status, ownership assignment, and attention category.

05 / CUSTOMER INFORMATION

Request setup data

Define global requests or requests tied to a template. Choose text, email, URL, number, or date input; mark requests required or optional; view response readiness.

Customer-provided values are workspace data and require deliberate privacy and retention controls.

06 / WORKSPACE TEAM

Members and invitations

Owners can create one-time invitation links for teammates; pending invitations can be revoked. Assign workspace members to process or implementation work.

Invitation links expire after seven days according to the product guide.

07 / SETTINGS

Appearance and account

Per-user light/dark mode and accent color preferences. Settings also provide account deletion with password confirmation and workspace ownership review.

Deleting a sole member deletes that workspace; a shared-workspace owner must transfer ownership before leaving.

08 / RAY

In-product Relay guide

Ray explains user-facing features and navigation from a curated product guide. Each account has its own saved conversation; users explicitly consent to external AI processing and can clear history.

Ray does not receive workspace records or perform product actions.

05 / END-TO-END WORKFLOW

From process knowledge to customer delivery

1. **Create a process map.** Name the process, describe its purpose, and create its steps in sequence.
2. **Make each step actionable.** Add instructions, a responsible role, an assignee when available, a stage, a due offset, and prerequisite steps.
3. **Run the process check.** Review findings about missing guidance, role/assignment and prerequisite/scheduling problems. Correct the process and check it again.
4. **Save a template snapshot.** Preserve the tested process as a reusable starting point. Version history supports review and restoration.
5. **Start a customer implementation.** Choose a template and customer name; Relay creates an independent customer-specific work plan.
6. **Configure customer information requests.** Choose scope, field type, and required/optional behavior. Keep the requested fields limited to legitimate setup needs.
7. **Share the customer link safely.** A customer with the link can submit requested fields. Rotate/revoke an old link when necessary.
8. **Track progress and unblock work.** Update task status, owner, dates and blocker details. Use prerequisites to clarify waiting work.
9. **Review readiness signals.** Check for missing/invalid required inputs, unassigned work, overdue or blocked tasks, and unmet prerequisites; then coordinate the next action.

> **Readiness meaning:** a Relay summary reports the state of data entered into Relay. It does not certify external system configuration, security, implementation quality, or customer launch approval.

First-use guidance

The workspace presents a dismissible, progress-aware first path: create a process map, add steps, check it, save a template, and start an implementation. Empty sections aim to explain the purpose and offer a next action rather than leaving a blank screen.

Who does what in Relay

Person	Typical activity	Access notes
Workspace owner	Sets up workspace, manages processes/templates, invites teammates, and leads customer implementations.	Can manage invitations. Ownership needs transfer if the owner deletes their account from a shared workspace.
Workspace member	Works on assigned process or customer tasks and helps manage implementations.	Cannot create or revoke invitations in the current role model.
Customer participant	Uses the shared private customer link to submit the named setup values.	Link-based access rather than a full customer account; links have no built-in expiry, so protect and rotate them to revoke access.
Relay operator	Handles sales inquiries, production configuration, incidents, data requests, and billing/support.	Real operator identity and staffed support channel are not yet established.

Account lifecycle

- Account creation uses email and a password with a 12-character minimum.
- Sign-in creates a server-side session represented by an HTTP-only cookie; session expiry is 14 days.
- Password recovery sends a one-time link through a configured email provider. Reset tokens are stored as hashes and expire after 30 minutes.
- Invitations are associated with an email and one-time token; invited users join via the product signup/sign-in flow.
- Account deletion requires the current password and an explicit typed confirmation, then clears the session cookie.

07 / INFORMATION MODEL

Data Relay stores and why

Relay uses SQLite. The table inventory below is from the repository schema; it describes categories only and contains no live account or customer values.

Data group	Stored entities	Information represented
Sales inquiries	`inquiries`	Name, phone, message, status, and creation time; company/email may be embedded in the inquiry message.
Identity and access	`product_users`, `product_workspaces`, `product_memberships`, `product_sessions`	Email, password hash, workspace name, owner/member membership, hashed session tokens, expiry.
Account support and settings	`product_password_resets`, `product_help_messages`, `product_help_consents`, `product_user_preferences`, `product_user_onboarding`	Hashed reset token, consented Ray messages, consent timestamp, theme/accent, onboarding dismissal.
Team	`product_workspace_invites`	Workspace, invited email, hashed invite token, expiry/acceptance, creator.
Process design	`product_processes`, `product_process_steps`, `product_process_step_dependencies`, `product_setup_checks`	Process metadata, ordered steps/instructions/owners/assignments/schedule, prerequisites, check results/findings.
Reusable knowledge	`product_templates`, `product_template_versions`	Versioned process snapshots and template metadata for future implementations.
Customer delivery	`product_implementations`, `product_implementation_steps`, `product_implementation_step_dependencies`	Customer name, hashed portal-link token, copied task plan, status, dates, blockers, next actions, prerequisites.
Customer information	`product_requirements`, `product_customer_inputs`	Request labels/instructions/type/scope/required flag and customer-submitted response values.

Data minimization: avoid collecting secrets or unrelated confidential content. The customer form warns against passwords, payment-card details, and secrets; operators should also set retention, access and deletion procedures appropriate to the actual deployment.

08 / TECHNICAL SYSTEM

Architecture, runtime, and integrations

Application

Node.js application using Express. The browser app is served as static HTML, CSS, and JavaScript; JSON endpoints implement authentication and product operations.

Persistence

SQLite through `better-sqlite3`, with foreign keys enabled and WAL journaling. Database schema setup includes additive column migrations.

Production container

Docker image based on Node 22 Bookworm slim. The runtime runs as the non-root `node` user, exposes port 3000, uses read-only root filesystem in Compose, and persists database files in a named volume.

Runtime health

`/healthz` and `/health` check a database query. The endpoint reports app/database availability; it is not external uptime monitoring.

Configured provider integrations

Integration	Use	Configuration / data flow
Email delivery provider	Password reset email	API key and verified sender are server configuration. Recovery link contains a one-time token.
Telegram Bot API	New contact/demo inquiry notification	Bot token and destination chat ID; inquiry remains stored even when notification delivery fails.
OpenAI-compatible chat completions API	Ray product-help answers	Server-side key/model. After explicit consent, the question and recent private conversation are sent; workspace records are not included as assistant context.
Reverse proxy / TLS	Public HTTPS, host routing, forwarded client information	Host must terminate HTTPS and trust proxy hops must match the real proxy path.

Not implemented: product-management integrations, billing/checkout, automatic customer system setup, infrastructure diagnostics, SAML/SSO, and customer-specific enterprise access integrations.

Existing protections and meaningful limits

Implemented safeguards

- Scrypt password hashes; password reset and invitation tokens are stored as hashes.
- HTTP-only, SameSite=Strict session cookie; Secure attribute in production.
- Same-origin checks on state-changing product routes.
- Session-required access checks and workspace-scoped records.
- Rate limits on account creation, sign-in, password recovery, account deletion, and Ray requests.
- Security headers include no-referrer, nosniff, frame denial, permissions policy, and production HSTS.
- Customer links store a token hash and can be rotated.

Limits to understand

- Rate-limit counters are in memory; they reset on restart and do not coordinate across instances.
- No email verification or multi-factor authentication was found in the reviewed account flow.
- Customer links are bearer links; anyone who obtains one may access that form until rotation/expiry controls apply.
- Consent to Ray means a third-party AI provider receives the question and recent chat, subject to that provider's terms.
- SQLite WAL requires appropriate persistent-volume and backup handling.
- No Content-Security-Policy was found in the reviewed server headers.
- External monitoring, support response, incident process, and restore procedure remain operator responsibilities.

Account deletion behavior

- Deletion is offered under Settings; the user reviews membership impact and confirms with password plus typed `DELETE`.
- A sole-member workspace is deleted with the user, including workspace/customer records linked by cascading foreign keys.
- For shared workspaces, the member is removed. An owner must assign a remaining member as owner first.
- Personal sessions, preferences, onboarding state, help history, and memberships are removed through user deletion and foreign-key cascades; assigned work is unassigned.
- Live-database deletion does not itself erase existing backups. Restore/replay controls need to be configured and tested.

Privacy-policy readiness: the repository text is a product-level draft. It must be reconciled with the actual operator, hosting location, AI/email/messaging provider settings, retention, applicable law, and customer agreement before real data is accepted.

What has been tested

Verification	Result in this review	What it tells us
`npm test`	**4 / 4 passed**	Backup utility, integrated product/workspace workflows, account deletion, and public page/inquiry availability pass in local tests.
`npm audit --omit=dev`	**0 reported**	No known advisories were reported for production dependencies at review time.
`git diff --check`	**Passed**	No diff whitespace errors at the verified commit state.
Docker build/smoke test	**Not verified**	Docker is not installed in the current environment; Actions status could not be retrieved via the connected API.
Live deployment and third-party integrations	**Not verified**	No chosen public host/domain, configured production secrets, or production smoke-test evidence was available.

Test coverage at a glance

- Product suite exercises account sessions, passwords, workspace isolation, process/template/implementation workflows, customer information, invitation, Ray consent/history, and the account-deletion flow.
- Deletion tests verify wrong-password rejection, ownership transfer, sole-member workspace removal, session invalidation, and assignee cleanup.
- Backup suite checks SQLite snapshot integrity and overwrite protection.
- Coverage is not a guarantee of absence of bugs, penetration-test result, load test, recovery drill, legal review, or production availability.

What a real launch requires

1. **Choose host and domain.** Configure public HTTPS, deployment logs, process restart, and durable storage for `/app/data`.
2. **Protect configuration.** Set unique strong secrets in a managed secret store. Never commit `.env` or expose provider keys to browser code.
3. **Validate email and proxy.** Verify the sender domain/address and set the trusted-proxy-hop count to the actual topology.
4. **Build and smoke test.** Confirm the GitHub Actions Docker build and production image smoke test succeed, or run the same checks in a Docker-enabled environment.
5. **Exercise workflows with dummy data.** Sign up, recover/reset a password, invite teammate, map/check/save process, start implementation, submit customer form, test workspace isolation, and verify inquiry notification.
6. **Prove persistence.** Restart the service, recheck health and sign-in, and confirm the SQLite database remains on persistent storage.
7. **Prepare recovery.** Schedule encrypted off-host backups, restrict access, test restoration, and keep a restricted deletion-replay process.
8. **Operate the service.** Establish monitored support, external uptime/log alerts, incident handling, retention, data requests, and owner responsibilities.
9. **Finalize customer terms.** Identify provider/jurisdiction, data handling roles, pricing, billing, renewal, cancellation, refund, liability and support terms in writing.

> **Current state:** the project has deployment instructions, Docker and Compose definitions, and a CI workflow. However, the local environment has no Docker executable and an empty local `.env`; no production host/domain is chosen. Therefore image, public HTTPS, secrets, email delivery, AI provider, notification, persistence, backups, and monitoring are unverified.

The deployment playbook is `DEPLOYMENT.md`. It specifies a 30-day maximum protected-backup lifetime and requires deletions to be replayed after restoring an older snapshot. A restricted deletion ledger and explicit restore exercise are operational dependencies, not automatic features.

Strengths, risks, and verdict

Strengths

- Clear, connected core implementation workflow.
- Actionable onboarding and empty states.
- Workspace isolation and regression tests.
- Typed customer inputs and inspectable readiness flags.
- Process snapshots and template history.
- Consent-based, user-guide-limited assistant.
- Account deletion and ownership transfer now represented in product.

Risks and unknowns

- No production deployment evidence or successful Docker run confirmed here.
- Provider/operator identity, jurisdiction, support contact, and customer agreement unsettled.
- No completed proof of backups, monitoring, recovery, or real provider integrations.
- Demand, pilot success, willingness to pay, and value metrics are unproven.
- No billing, SSO/MFA, external project sync, or mature enterprise administration.
- Data deletion from backups depends on a working restore/replay discipline.

Best next milestones

1. Keep the current product in synthetic-data demonstrations while business/legal arrangements are unsettled.
2. Verify the latest GitHub Actions result for commit e53033c and resolve any Docker build/smoke-test failure.
3. Pick the actual operating jurisdiction and an eligible contracting/payment operator; do not pick a jurisdiction to bypass age or provider rules.
4. Choose production host/domain, configure secrets and HTTPS, then test email, inquiry, AI, database persistence, and backup restoration.
5. Review privacy and service terms with the actual operator/provider and obtain appropriate local legal guidance before client data or payment.
6. Only after those gates, run a small, explicitly scoped pilot and measure onboarding friction, customer response effort, completion gaps, and recurring usage.

> **Final verdict:** Relay is a credible, functionally developed prototype for demos and an early pilot after the operational gates are completed. It is not yet demonstrated as a live, client-ready production service. This is a distinction between implemented product and unverified business/deployment readiness.

Repository map and glossary

Repository path	Role
`server.js`	Express startup, public inquiry endpoint, inquiry API, security headers, health check, error handling.
`product-routes.js`	Product API: authentication, workspaces, process maps, templates, implementations, customer forms, preferences, Ray, deletion.
`db.js`	SQLite schema, additive migrations, database connection and inquiry helpers.
`relay-help-content.js`	Curated Relay user guide and assistant scope/security instructions.
`public/product.html`, `product.js`, `product.css`	Authenticated product interface, app behavior, themes and visual design.
`public/customer.html`, `customer.js`	External customer setup-information form.
`public/index.html`, other public pages, `relay.css`	Marketing, demo, contact, pricing, privacy, terms, and inquiry dashboard.
`Dockerfile`, `docker-compose.yml`	Container image and single-service deployment setup.
`.github/workflows/ci.yml`	Automated tests, Compose validation, image build, container smoke checks.
`DEPLOYMENT.md`	Configuration, first deployment, backup/restore, and operator responsibilities.
`test/product.test.js`, `test/backup.test.js`	Product integration and database backup tests.

Glossary

- **Process map:** ordered reusable plan of stages and tasks.
- **Template:** versioned snapshot used as a starting point for future customer work.
- **Implementation:** customer-specific copy of template work.
- **Prerequisite:** task dependency that can leave later work waiting.
- **Readiness gap:** missing/invalid response or attention state surfaced by Relay.
- **Customer portal link:** private bearer URL for a customer information request.
- **Ray:** opt-in AI assistant limited to explaining Relay user-facing features.
- **Inquiry dashboard:** separate internal view of sales/demo contact submissions.
- **Workspace owner:** administrator role for invitations and workspace ownership.
- **Deletion replay:** applying post-backup deletions after restoring an earlier database copy.

Source basis: current Relay repository at commit e53033c, including product pages, server/API code, database schema, deployment files, and tests. The document describes repository evidence; unverified provider behavior, live data, and legal requirements are intentionally not asserted.